

Rogue Tower — Forensic Analysis & Solution Report

Complete technical walkthrough, protocol decodes, traffic attribution, and cryptanalysis

VERIFIED CAPTURE FLAG

picoCTF{r0gu3_c3ll_t0w3r_3a5d55b2}

1. Executive Summary & Findings Matrix

Network forensic analysis was performed on packet capture `rogue_tower.pcap` to investigate a simulated cellular Rogue Base Station (IMSI-Catcher / Stingray) interception attack. An unauthorized cell tower broadcast an unauthorized network advertisement, forced a victim cellular terminal to associate with it, leaked the subscriber identity (IMSI), and collected six encrypted HTTP POST exfiltration packets.

Forensic Parameter	Observed Value	Technical Significance
Rogue Tower Beacon Source	192.168.99.1 (UDP 55000)	Broadcast address sending unauthorized cellular network beacons
Rogue Cell Attributes	CELLID=91521 PLMN=00101	Non-standard test PLMN used to lure cellular subscribers
Rogue C2 / Web Server	198.51.100.102 (Port 80)	Target destination for HTTP registration and data exfiltration
Compromised Victim IP	10.100.101.252	Mobile handset connected to the rogue base station
Victim Subscriber IMSI	310410050746829	Disclosed in User-Agent header of Packet 16
Exfiltration Method	6 × HTTP POST /upload	Segmented base64 slices sent to 198.51.100.102
Reassembled Base64	RVlUW3VsdEJHAFBBBwdRCllcaEAGTwFLagNWAVINB1sHTQ==	34-byte encrypted binary payload
Encryption Scheme	Repeating 8-byte XOR	Symmetric XOR stream cipher
Recovered XOR Key	b"50746829" (ASCII: 50746829)	Key is the lower 8 digits (MSIN) of the victim's IMSI!
Recovered Flag	picoCTF{r0gu3_c3ll_t0w3r_3a5d55b2}	Decrypted plaintext payload

2. Phase 1: Identifying the Rogue Cell Tower

In cellular networks and SDR testbeds (such as OpenBTS or srsRAN), base stations broadcast System Information Blocks (SIB) to announce their Public Land Mobile Network (PLMN) identity and Cell ID. In `rogue_tower.pcap`, base station beacons are broadcasted over UDP port 55000.

Wireshark Display Filter

```
udp.port == 55000
```

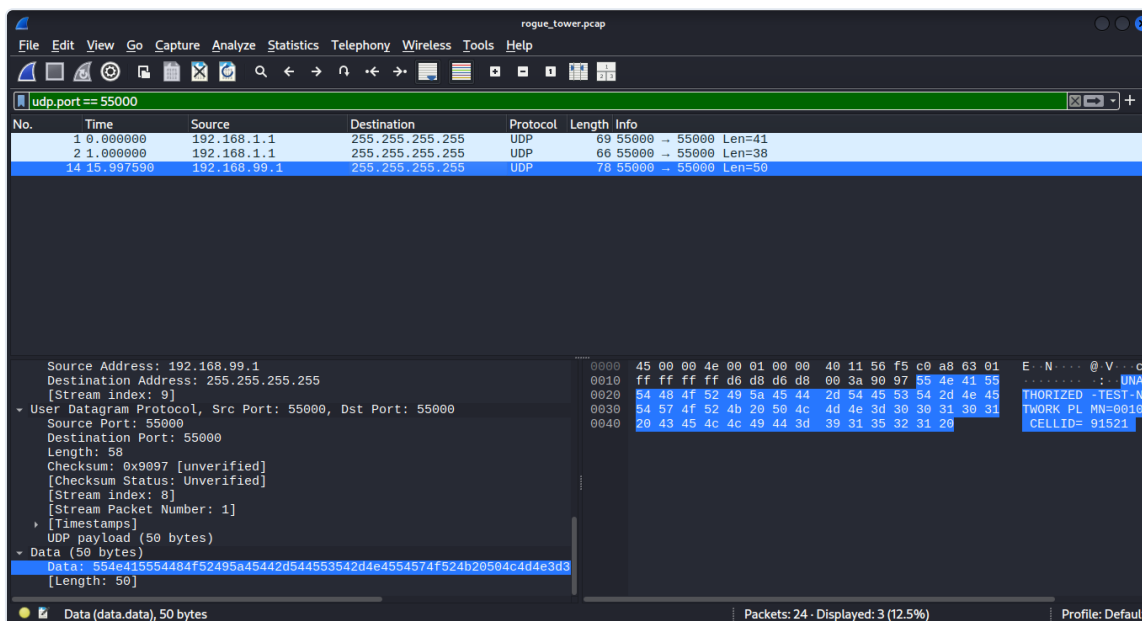


Figure 1: Wireshark capture filtered on `udp.port == 55000`, displaying Packet 14 beacon from 192.168.99.1 with payload UNAUTHORIZED-TEST-NETWORK PLMN=00101 CELLID=91521.

Packet Breakdown

- Packet 1 (t = 0.000s): 192.168.1.1 → 255.255.255.255 | CARRIER: Verizon PLMN=310410 CELLID=14884
- Packet 2 (t = 1.000s): 192.168.1.1 → 255.255.255.255 | CARRIER: AT&T PLMN=310410 CELLID=14885
- Packet 14 (t = 15.997s): 192.168.99.1 → 255.255.255.255 | UNAUTHORIZED-TEST-NETWORK PLMN=00101 CELLID=91521

Forensic Assessment: Rogue Base Station Detected

Legitimate cell towers operate under commercial US PLMN 310410 from 192.168.1.1. In contrast, Packet 14 is transmitted from a separate subnet (192.168.99.1) advertising an explicit UNAUTHORIZED-TEST-NETWORK with test PLMN 00101 and CELLID=91521.

3. Phase 2: Identifying the Compromised Device & IMSI

After the rogue cell starts broadcasting at `t = 15.997s`, mobile clients initiate registration traffic over HTTP. We apply the HTTP display filter to trace client associations:

Wireshark Display Filter

http

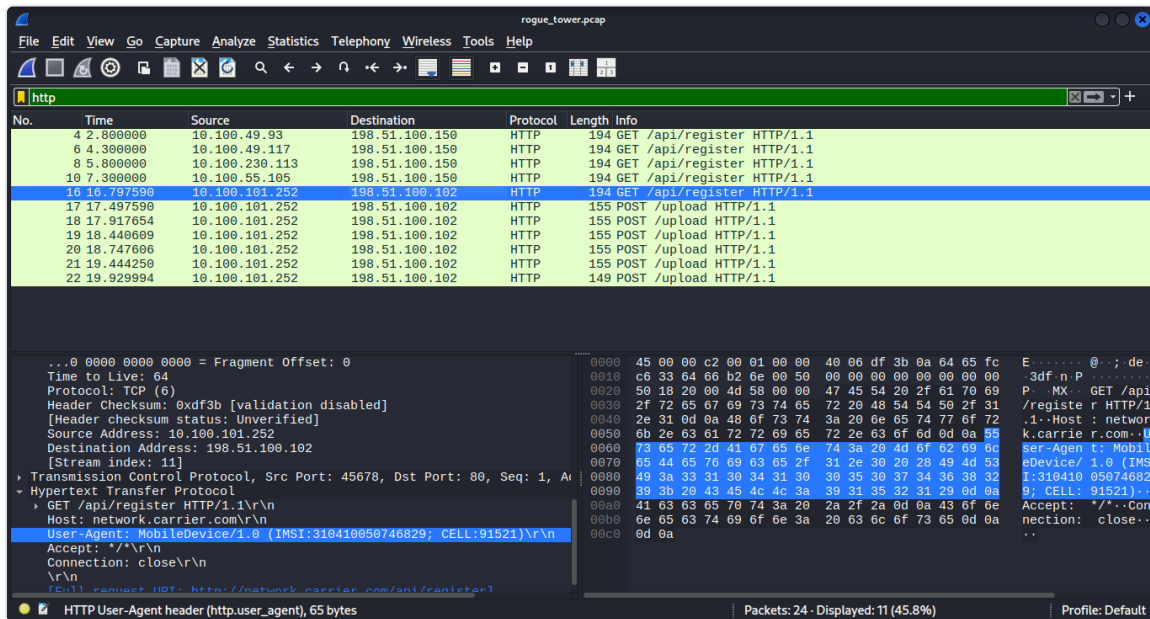


Figure 2: Wireshark HTTP traffic view highlighting Packet 16, where victim 10.100.101.252 registers to rogue server 198.51.100.102 transmitting its IMSI and Cell ID.

Traffic Attribution

- Legitimate Handsets (Packets 4, 6, 8, 10): IPs `10.100.49.93`, `10.100.49.117`, `10.100.230.113`, and `10.100.55.105` connect to the legitimate carrier registration server at `198.51.100.150` via `GET /api/register`.
- Compromised Victim (Packet 16): At `t = 16.797s`, IP `10.100.101.252` connects to the rogue collection server `198.51.100.102` via `GET /api/register HTTP/1.1`.

Inspection of Packet 16 Headers

```

GET /api/register HTTP/1.1
Host: network.carrier.com
User-Agent: MobileDevice/1.0 (IMSI:310410050746829; CELL:91521)
Accept: */*
Connection: close
  
```

Key Intelligence: Device & Subscriber Identification

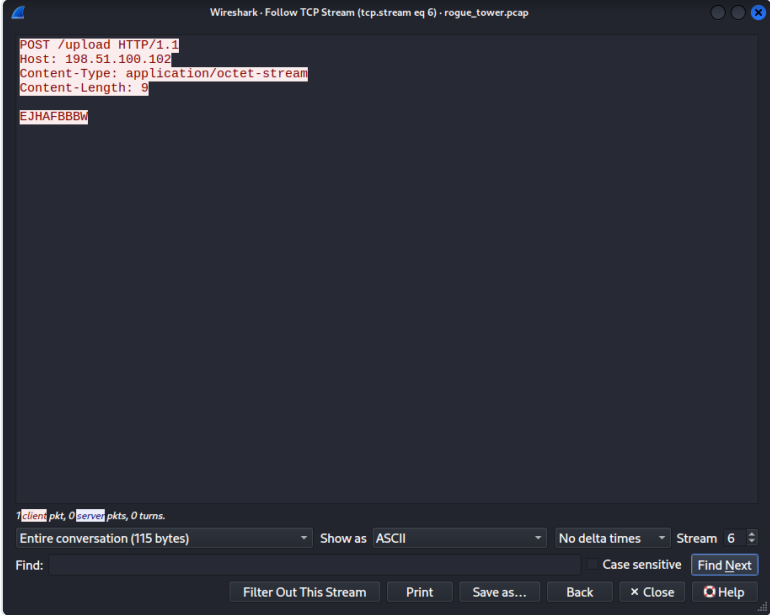
The User-Agent header explicitly links the victim IP 10.100.101.252 to the rogue cell CELL:91521. It also leaks the subscriber's International Mobile Subscriber Identity (IMSI): **310410050746829**.

4. Phase 3: Extracting Exfiltrated Data

Following initial registration, the compromised handset immediately executes six rapid HTTP POST requests to `http://198.51.100.102/upload` (Packets 17, 18, 19, 20, 21, 22).

Wireshark Display Filter

```
http.request.method == "POST" && ip.addr == 10.100.101.252
```



The screenshot shows the Wireshark 'Follow TCP Stream' window for Stream 6. The packet list on the left shows packet 18 selected. The packet details pane on the right shows the structure of the HTTP POST request, including the host (198.51.100.102), content type (application/octet-stream), and content length (9). The packet bytes pane shows the raw data EJHAFBBBW.

Figure 3: Wireshark “Follow TCP Stream” (Stream 6) showing Packet 18 transmitting exfiltrated slice EJHAFBBBW via POST /upload.

Payload Extraction via CLI (tshark)

```
tshark -r rogue_tower.pcap -Y "http.request.method == POST" -T fields -e frame.number -e http.file_data
```

Frame	Destination	Hex-Encoded Body	Decoded ASCII / Base64 Chunk
17	198.51.100.102	52566c555733567364	RVlUW3Vsd
18	198.51.100.102	454a48414642424257	EJHAFBBBW
19	198.51.100.102	6452436c6c63614541	dRClIcaEA
20	198.51.100.102	475477464c61674e57	GTwFLagNW
21	198.51.100.102	4156494e4231734854	AVINB1sHT
22	198.51.100.102	513d3d	Q==

Concatenating the chunks in order of transmission yields the complete base64 payload:

```
RVlUW3Vsd + EJHAFBBBW + dRClIcaEA + GTwFLagNW + AVINB1sHT + Q==
= RVlUW3VsdEJHAFBBBwdRClIcaEAGTwFLagNWAVINB1sHTQ==
```

5. Phase 4: Cryptanalysis & Script Breakdown (pap.py)

The file `/home/white/white/Music/PICO-CTF/picoctfAI/fornsicis/pap.py` solves the challenge by decrypting the exfiltrated ciphertext using a **Known-Plaintext Attack (KPA)** against a repeating XOR cipher.

Decryption Script Implementation

```
import base64

# 1. Base64-decode the reassembled exfiltration payload (34 bytes)
payload = base64.b64decode("RVlUW3VsdEJHAFBwRCl1caEAGTwFLagNWAVINB1sHTQ==")

# 2. Known plaintext prefix standard for picoCTF flags (8 bytes)
known = b"picoCTF{"

# 3. Known-Plaintext XOR attack: Key = Ciphertext ^ Plaintext
key = bytes(c ^ p for c, p in zip(payload[:len(known)], known))

# 4. Decrypt the full payload using the repeating 8-byte key
flag = bytes(c ^ key[i % len(key)] for i, c in enumerate(payload))

# 5. Output the recovered flag
print(flag.decode())
```

Mathematical Proof of the Known-Plaintext Attack

The XOR operation ($\oplus$) is commutative, associative, and its own inverse:

$$C[i] = P[i] \oplus K[i \bmod L] \Leftrightarrow K[i \bmod L] = C[i] \oplus P[i]$$

Where `C[i]` is the i-th ciphertext byte, `P[i]` is the known plaintext byte (`picoCTF{`), and `L = 8` is the key length.

Index i	Cipher Byte (C[i])	Known Plain Byte (P[i])	XOR Calculation (C[i] $\oplus$ P[i])	Derived Key Char
0	0x45	'p' (0x70)	0x45 $\wedge$ 0x70 = 0x35	'5'
1	0x59	'i' (0x69)	0x59 $\wedge$ 0x69 = 0x30	'0'
2	0x54	'c' (0x63)	0x54 $\wedge$ 0x63 = 0x37	'7'
3	0x5B	'o' (0x6f)	0x5B $\wedge$ 0x6f = 0x34	'4'
4	0x75	'C' (0x43)	0x75 $\wedge$ 0x43 = 0x36	'6'
5	0x6D	'T' (0x54)	0x6D $\wedge$ 0x54 = 0x38	'8'
6	0x74	'F' (0x46)	0x74 $\wedge$ 0x46 = 0x32	'2'
7	0x44	'{' (0x7b)	0x44 $\wedge$ 0x7b = 0x39	'9'

The Fundamental Cryptographic Link: Key $\equiv$ IMSI Suffix

The recovered 8-byte key is b"50746829".

Observe the structure of the victim subscriber's IMSI from Packet 16:

```
Victim IMSI:  3 1 0    4 1 0    0 0    [ 5 0 7 4 6 8 2 9 ]
              (MCC)   (MNC)         --- MSIN / Key ---
Derived Key:                5 0 7 4 6 8 2 9
```

The symmetric XOR key is literally the last 8 digits of the victim's IMSI! The author intended for either approach to succeed:

1. **Forensic Deduction:** Use the captured IMSI suffix directly as the decryption key.
2. **Known-Plaintext Cryptanalysis:** Derive the key from the `picoCTF{` prefix via `pap.py`.

6. Execution & Verification

Executing `pap.py` confirms the complete decryption:

```
$ python3 pap.py
picoCTF{r0gu3_c3ll_t0w3r_3a5d55b2}
```

7. Forensic Investigation Takeaways

- **Rogue BTS Signatures:** IMSI-catchers often broadcast abnormal PLMNs (e.g. `00101` test networks) and high transmit power to entice cell reselection.
- **Subscriber Identification Leakage:** Mobile protocols or insecure mobile app reporting channels can leak persistent identities (IMSI, IMEI) in unencrypted L7 traffic (such as HTTP User-Agent headers).
- **Weak Cryptography:** Repeating XOR keys derived from public or sniffable identifiers (such as IMSI/hardware serials) provide zero security against known-plaintext cryptanalysis.